

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança — Ginga

Versão auditada: 0.4.7 OVERLAY-FINAL-R1

Data: 01/09/2026

Escopo auditado

API Express/TypeScript, Prisma/PostgreSQL, autenticação e sessões, Web React/Vite, Desktop Electron, Socket.IO/LiveKit, Docker Compose/Dockerfiles, Caddy, GitHub Actions, scripts, documentação e variáveis de build do frontend.

Nota metodológica

Foram enumeradas 288 declarações de handlers HTTP. As cinco categorias solicitadas foram mapeadas para a stack detectada. A revisão é estática e baseada no código real do pacote fornecido; não substitui pentest dinâmico.

Limitação importante: o ZIP não contém diretório .git. O repositório público informado não pôde ser carregado/fetcheado neste ambiente; portanto o working tree foi escaneado, mas o histórico Git completo por segredos não foi verificado.

1. Stack detectada e mapeamento da auditoria

Camada	Stack verificada
Backend	TypeScript + Node.js 22 + Express 5.1.0
Banco/ORM	PostgreSQL 16 + Prisma 6.19.3; módulos v0.9 também usam SQL parametrizado via \$queryRawUnsafe/\$executeRawUnsafe
Auth	JWT HS256 (jsonwebtoken), tokenVersion, sessão revogável/lembrada, TOTP 2FA + recovery codes, bot/webhook tokens
Frontend	React 19.1.0 + Vite 7.1.1 + TypeScript
Realtime/media	Socket.IO 4.8.1 + LiveKit
Desktop	Electron 43.4.0 + electron-builder 26.15.3
Deploy/CI	Docker Compose + Dockerfiles + Caddy + GitHub Actions. Helm/Terraform não encontrados.
Isolamento de tenant	Sem RLS. Controle manual server-side por requireGuildMember/requireGuildCapability/requireChannelCapability e filtros guildId/userId.

Como cada categoria foi adaptada

1. Banco sem tranca

Sem Supabase/RLS. Mapeado para filtros/guards manuais por guildId/userId e integridade de referências entre Guild, Channel, Category, Role e tabelas v0.9.

2. Permissão no navegador

Gates React (systemRole, permissions.*, playbackOwner) foram cruzados com os endpoints de escrita/controle correspondentes no Express.

3. IDOR

Todas as 288 declarações de handlers HTTP do backend foram enumeradas; rotas por ID foram rastreadas até ownership/membership/guild/capability ou marcadas como achado.

4. Chaves expostas

Busca no working tree, .env.example, Compose, Dockerfiles, CI, scripts, docs, PEMs e variáveis VITE. Histórico Git não disponível no ZIP e fetch público falhou.

5. Inputs/XSS

Busca por innerHTML/dangerouslySetInnerHTML/eval/new Function, markdown/links, src/href controlados por usuário e HTML de e-mail; sinks encontrados foram rastreados até escaping/allowlist.

2. Resumo executivo

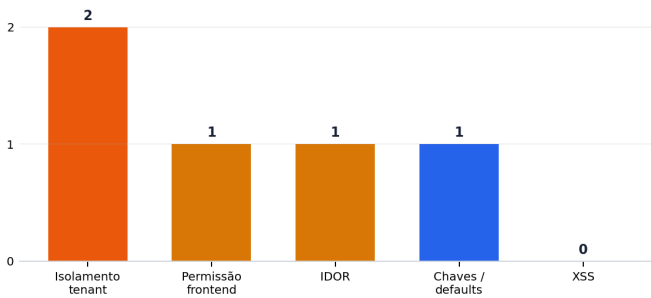
Foram confirmados 5 achados: 1 alto, 3 médios e 1 baixo. O risco central é a quebra de isolamento entre servidores em referências auxiliares da camada v0.9, incluindo um caso com impacto direto de escalada de permissão via onboarding.

CRÍTICA	ALTA	MÉDIA	BAIXA
0	1	3	1

Achados por severidade



Achados por categoria primária



Risco principal: algumas tabelas/rotas auxiliares validam o tenant do recurso pai, mas aceitam IDs relacionados sem verificar que pertencem ao mesmo guild. Isso contorna a camada de autorização centralizada que está correta na maior parte do produto.

3. Pontos fortes e pontos fracos

Pontos fortes verificados

Evidência	Controle verificado
Autenticação revogável apps/api/src/middleware.ts:8-30	Bearer JWT é validado e ainda é cruzado com tokenVersion, accountDisabled e sessão ativa.
Isolamento base de guild apps/api/src/permissions.ts:205-210	requireGuildMember usa chave composta guildId_userId e bloqueia acesso a quem não participa.
Autorização de canal apps/api/src/permissions.ts:383-457	requireChannelCapability centraliza membership, timeout, lockdown e overrides de role/usuário.
Mensagens e busca apps/api/src/routes/channels.ts:16-37; messages.ts:395-427	Leitura e busca de mensagens passam por visibilidade efetiva de canal antes da query final.
Cargos customizados apps/api/src/routes/roles.ts:187-236	Atribuição e overrides normais validam guild, hierarquia e vínculo channel/category-role.
DM/amizades apps/api/src/routes/social.ts:291-325	Aceite/remoção conferem a parte autenticada e listagem de DMs nasce das memberships do próprio usuário.
Uploads apps/api/src/routes/uploads.ts:73-149	Valida magic bytes, cota por uploader, delete por ownership e download por chave em comparação timing-safe + headers defensivos.
LiveKit apps/api/src/routes/livekit.ts:63-112	Token de voz requer connect no canal; token direto requer participação na chamada/conversa.
XSS/HTML apps/web/src/components/MessageContent.tsx:40-64; apps/web/src/lib/gamingProfile.ts:63-65,158-164	Links são limitados a http/https; mensagens usam React nodes; pontos de innerHTML encontrados aplicam escape explícito.
CI/repository gate .github/workflows/ci.yml:25-62; scripts/prepare-github.sh:58-65	CI compila API/Web e executa gate de repositório; há scanner simples de segredos hardcoded.

Pontos fracos centrais

- As extensões v0.9 criadas com SQL auxiliar não herdaram automaticamente a disciplina de tenant do Prisma principal.
- Algumas relações possuem FKs independentes sem invariante guildId + foreignId, deixando a aplicação responsável por todas as checagens.
- O Ginga Music trata ENDED como sinal confiável do player, mas o servidor não autentica a posse do lease.
- Credenciais de infraestrutura ainda dependem do operador executar init.sh em vez de falhar de forma fechada.

Cobertura das rotas

O backend contém 288 declarações de handlers HTTP em 17 arquivos de rotas/index. Cada declaração foi extraída com arquivo, linha, método, path, guards detectados e status de revisão. A matriz completa está em docs/security-audit/cobertura-rotas.csv.

4. Achados detalhados

Severidade	Arquivo:linha	Descrição
ALTA	apps/api/src/routes/v090.ts:76	F1 - Onboarding aceita cargo de outro servidor e o aplica como permissão local A criação de opções do onboarding aceita roleId e channelId arbitrários. Na conclusão do onboarding, o roleId é gravado em GuildMemberCustomRole com o guildId da vítima sem confirmar que o cargo pertence ao mesmo servidor. O cálculo de permissões confia nessa associação e agrega as permissões do cargo estrangeiro.
MÉDIA	apps/api/src/routes/v090.ts:96	F2 - Atribuição de badge permite escrever em badge de outro servidor A rota de atribuição exige manageRoles no guildId do path, mas insere badgeId e userId diretamente. Ela não confirma que a badge pertence ao guild do path nem que o usuário alvo é membro dele.
MÉDIA	apps/api/src/routes/v090.ts:54	F3 - Referências auxiliares v0.9 aceitam categorias/canais de outro tenant Várias estruturas auxiliares validam a permissão no recurso pai, porém não validam o tenant dos IDs referenciados: conteúdo de Space, modLogChannelId e categoryId de template de voz dinâmica. As tabelas armazenam FKs simples por ID e não possuem vínculo composto com guild_id.
MÉDIA	apps/web/src/components/GingaMusicPlayer.tsx:306	F4 - Ação ENDED do Ginga Music confia no playbackOwner do frontend O frontend só envia ENDED quando playbackOwnerRef.current é verdadeiro. No backend, porém, a ação ENDED ignora musicAccess e exige apenas que o chamador seja membro do servidor. Um cliente modificado pode chamar o endpoint diretamente e avançar a fila.
BAIXA	.env.example:7	F5 - Defaults públicos de PostgreSQL/Redis podem virar credenciais reais se init.sh for ignorado .env.example contém senhas públicas CHANGE_ME para PostgreSQL e Redis. O compose aceita esses valores e permite Redis sem autenticação quando REDIS_PASSWORD fica vazio. A validação de startup rejeita CHANGE_ME para JWT/LiveKit, mas não falha para credencial de banco/Redis.

F1. Onboarding aceita cargo de outro servidor e o aplica como permissão local

ALTA

Categoria: Banco sem tranca / isolamento de tenant

A criação de opções do onboarding aceita roleId e channelId arbitrários. Na conclusão do onboarding, o roleId é gravado em GuildMemberCustomRole com o guildId da vítima sem confirmar que o cargo pertence ao mesmo servidor. O cálculo de permissões confia nessa associação e agrega as permissões do cargo estrangeiro.

Por que é explorável

Explorável por um usuário com manageServer em um servidor A que conheça o ID de um cargo customizado de outro servidor B. Ao configurar uma opção do onboarding com esse ID e fazer um membro concluir o fluxo, as permissões do cargo de B passam a ser consideradas no servidor A. O atacante não precisa ser dono de B, apenas conhecer um roleId válido; a condição prática mais simples é participar de ambos os servidores.

Impacto

Escalada de privilégio entre tenants/servidores e quebra do isolamento de autorização.

Evidência linha por linha

apps/api/src/routes/v090.ts:76-76 - Entrada aceita roleId/channelIds sem validar o guild de cada referência.

```
76 | v090Router.post("/onboarding/questions/:questionId/options", requireAuth, asyncHandler(async (req, res) => {
    await ensureV090Storage();
    const q = routeParam(req.params.questionId, "questionId");
    qr = (await prisma.$queryRawUnsafe<any[]>(`SELECT guild_id FROM "GingaOnboardingQuestion" WHERE id=$1`, q))[0];
    if (!qr) throw new HttpError(404, "Pergunta nao encontrada");
    await requireGuildCapability(req.auth!.sub, qr.guild_id, "manageServer");
    const d = z.object({
      label: z.string().min(1).max(80),
      description: z.string().max(160).default(""),
      emoji: z.string().max(16).default(""),
      roleId: z.string().nullable().default(null),
      channelIds: z.array(z.string()).default([]),
      position: z.number().int().default(0)
    }).parse(req.body);
    id = randomUUID();
    await prisma.$executeRawUnsafe(`INSERT INTO "GingaOnboardingOption" (id, question_id, label, description, emoji, role_id, channel_ids, position) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)`, id, q, d.label, d.description, d.emoji, d.roleId, d.channelIds, d.position);
    res.status(201).json({ option: { id, ...d } });
  }));
```

apps/api/src/routes/v090.ts:81-81 - Conclusão grava roleId arbitrário em guildMemberCustomRole usando guildId do fluxo.

```
81 | v090Router.put("/guilds/:guildId/onboarding/me", requireAuth, asyncHandler(async (req, res) => {
    await ensureV090Storage();
    const g = routeParam(req.params.guildId, "guildId");
    answers = z.record(z.string(), z.array(z.string())).parse(req.body?.answers ?? {});
    await requireGuildMember(req.auth!.sub, g);
    const qs = await onboarding(g);
    roles = new Set<string>(), channels = new Set<string>();
    for (const q of qs.filter(q => q.enabled)) {
      const ids = answers[q.id] ?? [];
      if (q.required && !ids.length) throw new HttpError(400, `Resposta: ${q.title}`);
      for (const id of ids) {
        const o = q.options.find((x: any) => x.id === id);
        if (!o) throw new HttpError(400, "Opcao invalida");
        if (o.roleId) roles.add(o.roleId);
        for (const c of o.channelIds) channels.add(c);
      }
    }
    await prisma.$transaction(async tx => {
      for (const roleId of roles) await tx.guildMemberCustomRole.upsert({
        where: { guildId_userId_roleId: { guildId: g, userId: req.auth!.sub, roleId } },
        create: { guildId: g, userId: req.auth!.sub, roleId },
        for (const c of channels) await tx.$executeRawUnsafe(`INSERT INTO "GingaChannelPreference" (user_id, channel_id, favorite, hidden, updated_at) VALUES ($1, $2, TRUE, FALSE, NOW()) ON CONFLICT (user_id, channel_id) DO UPDATE SET favorite=TRUE, hidden=FALSE, updated_at=NOW()`, req.auth!.sub, c);
        await tx.$executeRawUnsafe(`INSERT INTO "GingaOnboardingProgress" (guild_id, user_id, answers, completed_at, updated_at) VALUES ($1, $2, $3::jsonb, NOW(), NOW()) ON CONFLICT (guild_id, user_id) DO UPDATE SET answers=$3::jsonb, completed_at=NOW(), updated_at=NOW()`, g, req.auth!.sub, JSON.stringify(answers));
      });
    }
    res.json({ completed: true, answers, favoriteChannelIds: [...channels] });
  }));
```

apps/api/src/v090Storage.ts:27-29 - FK do onboarding aponta para role por ID e channel_ids é TEXT[], sem vínculo de tenant.

```
27 | `CREATE TABLE IF NOT EXISTS "GingaOnboardingOption" (id TEXT PRIMARY KEY, question_id TEXT NOT NULL REFERENCES "GingaOnboardingQuestion" (id) ON DELETE CASCADE, label VARCHAR(80) NOT NULL, description VARCHAR(160) NOT NULL DEFAULT '', emoji VARCHAR(16) NOT NULL DEFAULT '', role_id TEXT REFERENCES "GuildCustomRole" (id) ON DELETE SET NULL, channel_ids TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[], position INTEGER NOT NULL DEFAULT 0, created_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP);
28 | `CREATE TABLE IF NOT EXISTS "GingaOnboardingProgress" (guild_id TEXT NOT NULL REFERENCES "Guild" (id) ON DELETE CASCADE, user_id TEXT NOT NULL REFERENCES "User" (id) ON DELETE CASCADE, answers JSONB NOT NULL DEFAULT '{}':::jsonb, completed_at TIMESTAMP(3), updated_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP, PRIMARY KEY (guild_id, user_id));
29 | `CREATE TABLE IF NOT EXISTS "GingaOnboardingRoleGrant" (guild_id TEXT NOT NULL REFERENCES "Guild" (id) ON DELETE CASCADE, user_id TEXT NOT NULL REFERENCES "User" (id) ON DELETE CASCADE, role_id TEXT NOT NULL REFERENCES "GuildCustomRole" (id) ON DELETE CASCADE, granted_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP, PRIMARY KEY (guild_id, user_id, role_id));
```

apps/api/prisma/schema.prisma:297-306 - GuildMemberCustomRole referencia Guild e GuildCustomRole de forma independente.

```
297 | model GuildMemberCustomRole {
298 |   guildId String
299 |   userId String
300 |   roleId String
301 |   guild Guild @relation(fields: [guildId], references: [id], onDelete: Cascade)
302 |   user User @relation(fields: [userId], references: [id], onDelete: Cascade)
303 |   role GuildCustomRole @relation(fields: [roleId], references: [id], onDelete: Cascade)
304 |   createdAt DateTime @default(now())
305 |
306 |   @@id([guildId, userId, roleId])
```

apps/api/src/permissions.ts:250-258 - Cálculo agrega role.permissions das associações sem conferir role.guildId.

```
250 | const assignments = await prisma.guildMemberCustomRole.findMany({
251 |   where: { guildId, userId },
252 |   include: { role: { select: { permissions: true } } }
253 | });
254 | const granted = new Set(assignments.flatMap((assignment) => assignment.role.permissions));
255 | const permissions = { ...base };
256 | for (const capability of customRolePermissionKeys) {
257 |   if (granted.has(capability)) permissions[capabilityField[capability]] = true;
258 | }
```

Correção recomendada

Validar roleId e cada channelId contra o guildId da pergunta antes de persistir; revalidar na aplicação do onboarding; em effectiveGuildPermissionsForUser, carregar somente cargos cujo role.guildId seja igual ao guildId da associação; adicionar invariante/constraint de aplicação para impedir associações cruzadas.

F2. Atribuição de badge permite escrever em badge de outro servidor

MÉDIA

Categoria: IDOR

A rota de atribuição exige manageRoles no guildId do path, mas insere badgId e userId diretamente. Ela não confirma que a badge pertence ao guild do path nem que o usuário alvo é membro dele.

Por que é explorável

Um moderador com manageRoles em A e conhecimento do ID de uma badge de B pode chamar a rota de A com badgId de B e associá-la a qualquer User existente. A remoção da mesma associação contém a checagem que está ausente no PUT, confirmando a inconsistência.

Impacto

Alteração não autorizada de dados pertencentes a outro tenant; integridade de badges e perfis afetada.

Evidência linha por linha

apps/api/src/routes/v090.ts:96-98 - PUT insere IDs sem vínculo; DELETE logo abaixo valida badge.id + guild_id.

```
96 | v090Router.put("/guilds/:guildId/badges/:badgeId/members/:userId", requireAuth, asyncHandler(async (req, res)
=>{const g=routeParam(req.params.guildId,"guildId");await requireGuildCapability(req.auth!.sub,g,"manageRole
s");await prisma.$executeRawUnsafe(`INSERT INTO "GingaGuildBadgeAssignment" (badge_id,user_id,assigned_by) VA
LUES($1,$2,$3) ON CONFLICT DO NOTHING`,routeParam(req.params.badgeId,"badgeId"),routeParam(req.params.userId
,"userId"),req.auth!.sub);res.status(204).end();}));
97 | v090Router.delete("/guilds/:guildId/badges/:badgeId", requireAuth, asyncHandler(async (req, res)=>{await ensu
reV090Storage();const g=routeParam(req.params.guildId,"guildId"),b=routeParam(req.params.badgeId,"badgeId");
await requireGuildCapability(req.auth!.sub,g,"manageRoles");const found=(await prisma.$queryRawUnsafe<any[]>
(`SELECT id FROM "GingaGuildBadge" WHERE id=$1 AND guild_id=$2`,b,g))[0];if(!found)throw new HttpError(404,"
Badge nao encontrada");await prisma.$executeRawUnsafe(`DELETE FROM "GingaGuildBadge" WHERE id=$1 AND guild_i
d=$2`,b,g);emit(req,g);res.status(204).end();}));
98 | v090Router.delete("/guilds/:guildId/badges/:badgeId/members/:userId", requireAuth, asyncHandler(async (req, r
es)=>{await ensureV090Storage();const g=routeParam(req.params.guildId,"guildId"),b=routeParam(req.params.bad
geId,"badgeId"),u=routeParam(req.params.userId,"userId");await requireGuildCapability(req.auth!.sub,g,"manag
eRoles");const found=(await prisma.$queryRawUnsafe<any[]>(`SELECT id FROM "GingaGuildBadge" WHERE id=$1 AND
guild_id=$2`,b,g))[0];if(!found)throw new HttpError(404,"Badge nao encontrada");await prisma.$executeRawUnsa
fe(`DELETE FROM "GingaGuildBadgeAssignment" WHERE badge_id=$1 AND user_id=$2`,b,u);emit(req,g);res.status(20
4).end();}));
```

apps/api/src/v090Storage.ts:34-35 - Assignment referencia badge_id/user_id, sem guild_id próprio ou constraint composta.

```
34 | `CREATE TABLE IF NOT EXISTS "GingaGuildBadge" (id TEXT PRIMARY KEY,guild_id TEXT NOT NULL REFERENCES "Gui
ld"(id) ON DELETE CASCADE,name VARCHAR(32) NOT NULL,icon VARCHAR(16) NOT NULL DEFAULT '',color VARCHAR(7) NO
T NULL DEFAULT '#7c3c9f',description VARCHAR(120) NOT NULL DEFAULT '',created_at TIMESTAMP(3) NOT NULL DEFAU
LT CURRENT_TIMESTAMP)`
35 | `CREATE TABLE IF NOT EXISTS "GingaGuildBadgeAssignment" (badge_id TEXT NOT NULL REFERENCES "GingaGuildBad
ge"(id) ON DELETE CASCADE,user_id TEXT NOT NULL REFERENCES "User"(id) ON DELETE CASCADE,assigned_by TEXT REF
ERENCES "User"(id) ON DELETE SET NULL,assigned_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,PRIMARY KE
Y(badge_id,user_id))`
```

Correção recomendada

Buscar badge por {id: badgId, guildId} antes do INSERT, exigir requireGuildMember(userId, guildId) para o alvo e manter a operação limitada à badge validada. Adicionar teste BOLA com IDs de guilds distintos.

F3. Referências auxiliares v0.9 aceitam categorias/canais de outro tenant

MÉDIA

Categoria: Banco sem tranca / isolamento de tenant

Várias estruturas auxiliares validam a permissão no recurso pai, porém não validam o tenant dos IDs referenciados: conteúdo de Space, modLogChannelId e categoryId de template de voz dinâmica. As tabelas armazenam FKs simples por ID e não possuem vínculo composto com guild_id.

Por que é explorável

Exige capacidade legítima de administração no guild A e conhecimento de IDs de guild B. Em Space, IDs estrangeiros podem ser anexados a A. Em voz dinâmica, uma categoria de B pode ser salva no template e utilizada na criação de um Channel com guildId=A/categoryId=B; como canais sincronizados herdam permissões da category relacionada, regras estruturais da categoria estrangeira podem influenciar o canal de A.

Impacto

Quebra de invariantes de tenant, corrupção de relações e possibilidade de aplicar regras de categoria de outro servidor a um canal recém-criado. O modLogChannelId atualmente é sobretudo um problema de integridade de configuração, pois o uso de envio de log não foi encontrado no código auditado.

Evidência linha por linha

apps/api/src/routes/v090.ts:54-54 - Space content insere categoryIds/channelIds após validar só o guild do Space.

```
54 | v090Router.put("/spaces/:spaceId/content", requireAuth, asyncHandler(async (req, res) => {
    await ensureV090Storage();
    const id = routeParam(req.params.spaceId, "spaceId"), rows = await prisma.$queryRawUnsafe<any[]>(`SELECT guild_id FROM "GingaGuildSpace" WHERE id=$1`, id);
    if (!rows[0]) throw new HttpError(404, "Area nao encontrada");
    const g = rows[0].guild_id;
    await requireGuildCapability(req.auth!.sub, g, "manageChannels");
    const d = z.object({
      categoryIds: z.array(z.string()).max(100),
      channelIds: z.array(z.string()).max(300)
    }).parse(req.body);
    await prisma.$transaction(async tx => {
      await tx.$executeRawUnsafe(`DELETE FROM "GingaGuildSpaceCategory" WHERE space_id=$1`, id);
      await tx.$executeRawUnsafe(`DELETE FROM "GingaGuildSpaceChannel" WHERE space_id=$1`, id);
      for (const c of [...new Set(d.categoryIds)])
        await tx.$executeRawUnsafe(`INSERT INTO "GingaGuildSpaceCategory" (space_id, category_id) VALUES ($1, $2)`, id, c);
      for (const c of [...new Set(d.channelIds)])
        await tx.$executeRawUnsafe(`INSERT INTO "GingaGuildSpaceChannel" (space_id, channel_id) VALUES ($1, $2)`, id, c);
    });
    emit(req, g);
    res.json(d);
  }));
```

apps/api/src/v090Storage.ts:18-21 - Join tables possuem FKs simples e não codificam guild_id.

```
18 | `CREATE TABLE IF NOT EXISTS "GingaGuildSpaceCategory" (space_id TEXT NOT NULL REFERENCES "GingaGuildSpace" (id) ON DELETE CASCADE, category_id TEXT NOT NULL REFERENCES "ChannelCategory" (id) ON DELETE CASCADE, PRIMARY KEY(space_id, category_id))`,
19 | `CREATE UNIQUE INDEX IF NOT EXISTS "GingaGuildSpaceCategory_category_key" ON "GingaGuildSpaceCategory" (category_id)`,
20 | `CREATE TABLE IF NOT EXISTS "GingaGuildSpaceChannel" (space_id TEXT NOT NULL REFERENCES "GingaGuildSpace" (id) ON DELETE CASCADE, channel_id TEXT NOT NULL REFERENCES "Channel" (id) ON DELETE CASCADE, PRIMARY KEY(space_id, channel_id))`,
21 | `CREATE UNIQUE INDEX IF NOT EXISTS "GingaGuildSpaceChannel_channel_key" ON "GingaGuildSpaceChannel" (channel_id)`,
```

apps/api/src/routes/v090.ts:86-86 - security-policy persiste modLogChannelId sem conferir o guild do canal.

```
86 | v090Router.put("/guilds/:guildId/security-policy", requireAuth, asyncHandler(async (req, res) => {
    await ensureV090Storage();
    const g = routeParam(req.params.guildId, "guildId"), d = secSchema.parse(req.body);
    await requireGuildCapability(req.auth!.sub, g, "manageAutoMod");
    await prisma.$executeRawUnsafe(`INSERT INTO "GingaGuildSecurityPolicy" (guild_id, anti_raid_enabled, join_window_seconds, join_limit, quarantine_enabled, quarantine_minutes, new_account_hours, block_external_links, block_invites, max_mentions, duplicate_limit, require_moderation_reason, auto_timeout_minutes, mod_log_channel_id, updated_at) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, NOW()) ON CONFLICT(guild_id) DO UPDATE SET anti_raid_enabled=$2, join_window_seconds=$3, join_limit=$4, quarantine_enabled=$5, quarantine_minutes=$6, new_account_hours=$7, block_external_links=$8, block_invites=$9, max_mentions=$10, duplicate_limit=$11, require_moderation_reason=$12, auto_timeout_minutes=$13, mod_log_channel_id=$14, updated_at=NOW()`, g, d.antiRaidEnabled, d.joinWindowSeconds, d.joinLimit, d.quarantineEnabled, d.quarantineMinutes, d.newAccountHours, d.blockExternalLinks, d.blockInvites, d.maxMentions, d.duplicateLimit, d.requireModerationReason, d.autoTimeoutMinutes, d.modLogChannelId);
    res.json({policy: d});
  }));
```

apps/api/src/v090Storage.ts:30-31 - Policy/template referenciam Channel/Category apenas por ID.

```
30 | `CREATE TABLE IF NOT EXISTS "GingaGuildSecurityPolicy" (guild_id TEXT PRIMARY KEY REFERENCES "Guild" (id) ON DELETE CASCADE, anti_raid_enabled BOOLEAN NOT NULL DEFAULT FALSE, join_window_seconds INTEGER NOT NULL DEFAULT 0, join_limit INTEGER NOT NULL DEFAULT 0, quarantine_enabled BOOLEAN NOT NULL DEFAULT FALSE, quarantine_minutes INTEGER NOT NULL DEFAULT 0, new_account_hours INTEGER NOT NULL DEFAULT 0, block_external_links BOOLEAN NOT NULL DEFAULT FALSE, block_invites BOOLEAN NOT NULL DEFAULT FALSE, max_mentions INTEGER NOT NULL DEFAULT 0, duplicate_limit INTEGER NOT NULL DEFAULT 0, require_moderation_reason TEXT NOT NULL DEFAULT '', auto_timeout_minutes INTEGER NOT NULL DEFAULT 0, mod_log_channel_id TEXT NOT NULL REFERENCES "Channel" (id) ON DELETE CASCADE, updated_at TEXT NOT NULL DEFAULT NOW())`,
31 | `CREATE UNIQUE INDEX IF NOT EXISTS "GingaGuildSecurityPolicy_guild_key" ON "GingaGuildSecurityPolicy" (guild_id)`,
```



```

    ULT 30,join_limit INTEGER NOT NULL DEFAULT 8,quarantine_enabled BOOLEAN NOT NULL DEFAULT FALSE,quarantine_mi
    nutes INTEGER NOT NULL DEFAULT 10,new_account_hours INTEGER NOT NULL DEFAULT 24,block_external_links BOOLEAN
    NOT NULL DEFAULT FALSE,block_invites BOOLEAN NOT NULL DEFAULT FALSE,max_mentions INTEGER NOT NULL DEFAULT 8
    ,duplicate_limit INTEGER NOT NULL DEFAULT 5,require_moderation_reason BOOLEAN NOT NULL DEFAULT FALSE,auto_t
    imeout_minutes INTEGER NOT NULL DEFAULT 0,mod_log_channel_id TEXT REFERENCES "Channel"(id) ON DELETE SET NULL
    ,updated_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP`),
31 | `CREATE TABLE IF NOT EXISTS "GingaDynamicVoiceTemplate" (id TEXT PRIMARY KEY,guild_id TEXT NOT NULL REFER
    ENCES "Guild"(id) ON DELETE CASCADE,name VARCHAR(48) NOT NULL,name_pattern VARCHAR(48) NOT NULL DEFAULT 'Sal
    a de {user}',category_id TEXT REFERENCES "ChannelCategory"(id) ON DELETE SET NULL,user_limit INTEGER NOT NUL
    L DEFAULT 0,owner_controls BOOLEAN NOT NULL DEFAULT TRUE,auto_delete BOOLEAN NOT NULL DEFAULT TRUE,enabled B
    OOLEAN NOT NULL DEFAULT TRUE,position INTEGER NOT NULL DEFAULT 0,created_at TIMESTAMP(3) NOT NULL DEFAULT CU
    RRENT_TIMESTAMP,updated_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP`),

```

apps/api/src/routes/v090.ts:90-92 - Template aceita categoryId e create usa esse valor para criar canal no guild do path.

```

90 | v090Router.post("/guilds/:guildId/dynamic-voice/templates",requireAuth,asyncHandler(async(req,res)=>{await
    ensureV090Storage();const g=routeParam(req.params.guildId,"guildId");await requireGuildCapability(req.auth
    !.sub,g,"manageChannels");const d=z.object({name:z.string().trim().min(1).max(48),namePattern:z.string().tri
    m().min(1).max(48).default("Sala de {user}"),categoryId:z.string().nullable().default(null),userLimit:z.numb
    er().int().min(0).max(99).default(0),ownerControls:z.boolean().default(true),autoDelete:z.boolean().default(
    true),enabled:z.boolean().default(true),position:z.number().int().default(0)}).parse(req.body),id=randomUUID
    ());await prisma.$executeRawUnsafe(`INSERT INTO "GingaDynamicVoiceTemplate"(id,guild_id,name,name_pattern,cat
    egory_id,user_limit,owner_controls,auto_delete,enabled,position) VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)`),id,
    g,d.name,d.namePattern,d.categoryId,d.userLimit,d.ownerControls,d.autoDelete,d.enabled,d.position);res.statu
    s(201).json({template:{id,guildId:g,...d}});});
91 | v090Router.delete("/guilds/:guildId/dynamic-voice/templates/:templateId",requireAuth,asyncHandler(async(r
    eq,res)=>{await ensureV090Storage();const g=routeParam(req.params.guildId,"guildId"),t=routeParam(req.params
    .templateId,"templateId");await requireGuildCapability(req.auth!.sub,g,"manageChannels");const found=(await
    prisma.$queryRawUnsafe<any[]>(`SELECT id FROM "GingaDynamicVoiceTemplate" WHERE id=$1 AND guild_id=$2`,t,g))
    [0];if(!found)throw new HttpError(404,"Modelo nao encontrado");await prisma.$executeRawUnsafe(`DELETE FROM "
    GingaDynamicVoiceTemplate" WHERE id=$1 AND guild_id=$2`,t,g);emit(req,g);res.status(204).end();});
92 | v090Router.post("/guilds/:guildId/dynamic-voice/:templateId/create",requireAuth,asyncHandler(async(req,re
    s)=>{await ensureV090Storage();const g=routeParam(req.params.guildId,"guildId"),t=routeParam(req.params.tem
    plateId,"templateId");await requireGuildMember(req.auth!.sub,g);const r=(await prisma.$queryRawUnsafe<any[]>(`
    SELECT * FROM "GingaDynamicVoiceTemplate" WHERE id=$1 AND guild_id=$2 AND enabled=TRUE`,t,g))[0];if(!r)thro
    w new HttpError(404,"Modelo indisponivel");const user=await prisma.user.findUniqueOrThrow({where:{id:req.aut
    h!.sub},select:{displayName:true,username:true}},name=r.name_pattern.replace(/\{user\}/gi,user.displayName|
    {user.username}).slice(0,48),last=await prisma.channel.aggregate({where:{guildId:g,categoryId:r.category_id},
    _max:{position:true}},channel=await prisma.channel.create({data:{guildId:g,categoryId:r.category_id,name,ty
    pe:"VOICE",position:(last._max.position??-1)+1,syncPermissionsWithCategory:Boolean(r.category_id),permission
    s:{create:[{role:"MODERATOR",canView:true,canSendMessages:true,canConnect:true},{role:"MEMBER",canView:true,
    canSendMessages:true,canConnect:true}]}));await prisma.$executeRawUnsafe(`INSERT INTO "GingaDynamicVoiceRoo
    m"(channel_id,template_id,owner_user_id,user_limit) VALUES($1,$2,$3,$4)`,channel.id,t,req.auth!.sub,r.user_l
    imit);emit(req,g);res.status(201).json({channel,room:{ownerUserId:req.auth!.sub,userLimit:r.user_limit,locke
    d:false}});});

```

apps/api/prisma/schema.prisma:359-370 - Channel permite guildId e categoryId como FKs independentes.

```

359 | model Channel {
360 |   id                String           @id @default(cuid())
361 |   guildId           String
362 |   categoryId         String?
363 |   name              String
364 |   type              ChannelType
365 |   topic             String           @default("") @db.VarChar(1024)
366 |   slowModeSeconds    Int             @default(0)
367 |   position          Int             @default(0)
368 |   syncPermissionsWithCategory Boolean @default(true)
369 |   guild             Guild           @relation(fields: [guildId], references: [id], onDelete
    : Cascade)
370 |   category          ChannelCategory? @relation(fields: [categoryId], references: [id], onDelete: SetNull)

```

apps/api/src/permissions.ts:420-440 - Canal sincronizado usa permissões/custom overrides/user overrides da category relacionada.

```

420 | const permissionSource = channel.category && channel.syncPermissionsWithCategory
421 |   ? channel.category.permissions
422 |   : channel.permissions;
423 | const permission = permissionSource.find((item) => item.role === membership.role);
424 |
425 | let allowed = capability === "view"
426 |   ? permission?.canView ?? true
427 |   : capability === "sendMessages"
428 |     ? (permission?.canView ?? true) && (permission?.canSendMessages ?? true)
429 |     : (permission?.canView ?? true) && (permission?.canConnect ?? true);
430 |
431 | const roleIds = new Set(assignments.map((item) => item.roleId));
432 | const customSource = channel.category && channel.syncPermissionsWithCategory
433 |   ? channel.category.customRolePermissions

```

```

434 |     : channel.customRolePermissions;
435 |     const relevant = customSource.filter((item) => roleIds.has(item.roleId));
436 |     const override = resolveCustomOverride(relevant, capability);
437 |     if (override !== null) allowed = override;
438 |     const individualSource = channel.category && channel.syncPermissionsWithCategory ? channel.category.use
      rPermissions : channel.userPermissions;
439 |     const individual = individualSource.find((item) => item.userId === userId);
440 |     const individualField = capability === "view" ? individual?.canView : capability === "sendMessages" ? i
      ndividual?.canSendMessages : individual?.canConnect;

```

Correção recomendada

Criar helpers de validação tenant-scoped (`assertCategoryInGuild/assertChannelInGuild`) e usá-los em todos os IDs auxiliares antes de persistir. Para estruturas novas, preferir guardar `guild_id` e usar constraints/índices compostos quando o modelo permitir. Na criação dinâmica, revalidar `category_id` antes do `prisma.channel.create`.

F4. Ação ENDED do Ginga Music confia no playbackOwner do frontend

MÉDIA

Categoria: Permissão definida no navegador

O frontend só envia ENDED quando `playbackOwnerRef.current` é verdadeiro. No backend, porém, a ação ENDED ignora `musicAccess` e exige apenas que o chamador seja membro do servidor. Um cliente modificado pode chamar o endpoint diretamente e avançar a fila.

Por que é explorável

Qualquer membro autenticado do guild consegue consultar o estado/track atual e enviar POST `/music/control` com `action=ENDED` e `expectedTrackId` correto. Isso funciona mesmo quando `musicAllowMembers=false` e o cliente não possui o lease oficial de playback.

Impacto

Manipulação não autorizada da fila e negação de serviço da reprodução compartilhada.

Evidência linha por linha

`apps/web/src/components/GingaMusicPlayer.tsx:306-317` - Gate de `playbackOwner` existe somente no cliente antes do POST.

```

306 |     const reportEnded = useCallback((trackId: string) => {
307 |       if (!guildId || !trackId || endedRequestRef.current === trackId || !playbackOwnerRef.current) return;
308 |       endedRequestRef.current = trackId;
309 |       void api<{ state: MusicState }>(`api/guilds/${guildId}/music/control`, {
310 |         method: "POST",
311 |         body: JSON.stringify({ action: "ENDED", expectedTrackId: trackId })
312 |       }).then((result) => acceptState(result.state)).catch(() => undefined).finally(() => {
313 |         window.setTimeout(() => {
314 |           if (endedRequestRef.current === trackId) endedRequestRef.current = "";
315 |         }, 700);
316 |       });
317 |     }, [acceptState, guildId]);

```

`apps/api/src/routes/music.ts:178-184` - `musicAccess` é o helper server-side que restringe controle quando membros não podem controlar.

```

178 | async function musicAccess(userId: string, guildId: string, requireManager = false) {
179 |   const { membership, permissions } = await effectiveGuildPermissionsForUser(userId, guildId);
180 |   const settings = await guildMusicSettings(guildId);
181 |   const manager = membership.role === "OWNER" || membership.role === "ADMIN" || permissions.canManageServ
      er || permissions.canManageBots;
182 |   if (requireManager && !manager) throw new HttpError(403, "Voce nao pode gerenciar o Ginga Music neste s
      ervidor");
183 |   if (!requireManager && !settings.musicAllowMembers && !manager) throw new HttpError(403, "Somente a mod
      eracao pode controlar a musica neste servidor");
184 |   return { membership, permissions, settings, manager };

```

`apps/api/src/routes/music.ts:444-453` - Qualquer membro pode ler o estado e obter o track atual.

```

444 | musicRouter.get("/guilds/:guildId/music", requireAuth, asyncHandler(async (req, res) => {

```

```

445 |   const guildId = routeParam(req.params.guildId, "guildId");
446 |   const userId = req.auth!.sub;
447 |   await requireGuildMember(userId, guildId);
448 |   const settings = await guildMusicSettings(guildId);
449 |   const state = await runtimeFor(guildId);
450 |   res.json({
451 |     settings: publicSettings(settings),
452 |     state: publicRuntime(state)
453 |   });

```

apps/api/src/routes/music.ts:596-611 - ENDED usa apenas requireGuildMember e avança a fila quando expectedTrackId confere.

```

596 |   musicRouter.post("/guilds/:guildId/music/control", requireAuth, musicControlLimiter, asyncHandler(async (
      req, res) => {
597 |     const guildId = routeParam(req.params.guildId, "guildId");
598 |     const userId = req.auth!.sub;
599 |     const data = controlSchema.parse(req.body);
600 |     // ENDED e um sinal do player oficial, não um comando de moderação.
601 |     // Qualquer membro conectado pode reportar o fim da faixa, mas sempre precisa
602 |     // identificar a faixa atual; os demais comandos respeitam musicAllowMembers.
603 |     const settings = data.action === "ENDED"
604 |       ? await (async () => { await requireGuildMember(userId, guildId); return guildMusicSettings(guildId);
        })()
605 |       : (await musicAccess(userId, guildId)).settings;
606 |     if (!settings.musicEnabled) throw new HttpError(409, "O Ginga Music está desativado neste servidor");
607 |     const state = await runtimeFor(guildId);
608 |
609 |     if (data.action === "ENDED") {
610 |       if (!data.expectedTrackId || state.queue[0]?.id !== data.expectedTrackId) return res.json({ state: pu
        blicRuntime(state) });
611 |       advance(state, true);

```

Correção recomendada

Vincular ENDED ao lease de playback validado no servidor. Exigir clientId + lease/token não forjável e confirmar que o chamador possui o lease ativo para o guild/canal; alternativamente, mover avanço automático para um coordenador server-side. Não usar um booleano do frontend como fronteira de autorização.

F5. Defaults públicos de PostgreSQL/Redis podem virar credenciais reais se init.sh for ignorado

BAIXA

Categoria: Chaves expostas / defaults inseguros

.env.example contém senhas públicas CHANGE_ME para PostgreSQL e Redis. O compose aceita esses valores e permite Redis sem autenticação quando REDIS_PASSWORD fica vazio. A validação de startup rejeita CHANGE_ME para JWT/LiveKit, mas não falha para credencial de banco/Redis.

Por que é explorável

Condição necessária: operador copiar .env.example ou limpar REDIS_PASSWORD sem executar scripts/init.sh, ou expor essas redes/portas por configuração adicional. No compose padrão PostgreSQL/Redis não têm portas publicadas no host, reduzindo a exposição para rede interna/lateral. scripts/init.sh mitiga o cenário normal ao gerar segredos aleatórios, mas é uma convenção e não um controle fail-closed.

Impacto

Credenciais previsíveis/ausência de senha em serviços internos se o bootstrap seguro for contornado.

Evidência linha por linha

.env.example:7-8 - Placeholders públicos podem ser copiados como credenciais de runtime.

```

7 | POSTGRES_PASSWORD=CHANGE_ME_DATABASE_PASSWORD
8 | REDIS_PASSWORD=CHANGE_ME_REDIS_PASSWORD

```

docker-compose.yml:10-12 - PostgreSQL recebe POSTGRES_PASSWORD diretamente do ambiente.

```
10 |     POSTGRES_DB: ${POSTGRES_DB}
11 |     POSTGRES_USER: ${POSTGRES_USER}
12 |     POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

docker-compose.yml:22-33 - Redis aceita REDIS_PASSWORD vazio e inicia sem --requirepass nesse caso.

```
22 |     redis:
23 |       image: redis:7-alpine
24 |       restart: unless-stopped
25 |       security_opt:
26 |         - no-new-privileges:true
27 |       environment:
28 |         REDIS_PASSWORD: ${REDIS_PASSWORD:-}
29 |       command: ["/bin/sh", "-ec", 'if [ -n "$$REDIS_PASSWORD" ]; then exec redis-server --appendonly yes --requirepass "$$REDIS_PASSWORD"; else exec redis-server --appendonly yes; fi']
30 |       volumes:
31 |         - redis_data:/data
32 |       healthcheck:
33 |         test: ["CMD-SHELL", 'if [ -n "$$REDIS_PASSWORD" ]; then redis-cli --no-auth-warning -a "$$REDIS_PASSWORD" ping; else redis-cli ping; fi']
```

docker-compose.yml:89-94 - API constrói DATABASE_URL com POSTGRES_PASSWORD fornecido.

```
89 |     env_file: .env
90 |     environment:
91 |       NODE_ENV: production
92 |       PORT: 3001
93 |       DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}?schema=public
94 |       UPLOAD_DIR: /data/uploads
```

apps/api/src/config.ts:40-49 - Fail-closed cobre JWT/LiveKit, mas não PostgreSQL/Redis.

```
40 | }).superRefine((value, ctx) => {
41 |   if (/CHANGE_ME/i.test(value.JWT_SECRET)) ctx.addIssue({ code: z.ZodIssueCode.custom, path: ["JWT_SECRET"], message: "JWT_SECRET padrao nao e permitido" });
42 |   if (/CHANGE_ME/i.test(value.LIVEKIT_API_KEY)) ctx.addIssue({ code: z.ZodIssueCode.custom, path: ["LIVEKIT_API_KEY"], message: "LIVEKIT_API_KEY padrao nao e permitido" });
43 |   if (/CHANGE_ME/i.test(value.LIVEKIT_API_SECRET)) ctx.addIssue({ code: z.ZodIssueCode.custom, path: ["LIVEKIT_API_SECRET"], message: "LIVEKIT_API_SECRET padrao nao e permitido" });
44 | });
45 |
46 | const parsed = schema.safeParse(process.env);
47 | if (!parsed.success) {
48 |   console.error("Configuracao invalida:", parsed.error.flatten().fieldErrors);
49 |   process.exit(1);
```

scripts/init.sh:34-39 - Mitigação existente: bootstrap gera senhas/segredos aleatórios.

```
34 | replace_value POSTGRES_PASSWORD "${random_hex 24}"
35 | replace_value REDIS_PASSWORD "${random_hex 24}"
36 | replace_value JWT_SECRET "${random_hex 48}"
37 | replace_value MFA_ENCRYPTION_KEY "${random_hex 32}"
38 | replace_value LIVEKIT_API_KEY "lk_${random_hex 8}"
39 | replace_value LIVEKIT_API_SECRET "${random_hex 32}"
```

Correção recomendada

Tornar POSTGRES_PASSWORD e REDIS_PASSWORD obrigatórios no compose com \${VAR:?required}; iniciar Redis sempre com --requirepass; adicionar preflight/startup que rejeite vazio/CHANGE_ME para credenciais de infraestrutura. Manter init.sh como conveniência, não como única barreira.

5. Categoria sem achado explorável: XSS

Nenhum XSS explorável foi confirmado nos sinks revisados.

Ponto	Evidência	Resultado
Mensagens/markdown	MessageContent.tsx:7,40-64,125-139	React nodes; links reconhecidos apenas como http/https e revalidados antes de abrir.
Gaming profile innerHTML	gamingProfile.ts:63-65,158-164,186-242,407	Campos interpolados passam por escapeHtml; elementos fixos são template local.
Overlay Electron	game-overlay.html:30-61	Strings de jogo/canal/participante/avatar passam por esc antes de innerHTML.
E-mails	auth.ts:228-235,291-306; registrationVerification.ts:108-127	displayName, URL e código são escapados antes de HTML.
eval/new Function/dangerouslySetInnerHTML	Busca global em apps/web/src, apps/desktop/src e apps/api/src	Nenhuma ocorrência de eval/new Function/dangerouslySetInnerHTML.

Observação: não foi encontrada biblioteca dedicada de sanitização (ex.: DOMPurify). Isso não foi reportado como falha porque os sinks encontrados têm escape/allowlist verificável; uma lib dedicada pode ser adotada como defesa em profundidade em futuras telas que renderizem HTML arbitrário.

6. Recomendações priorizadas

Prioridade	Ação	Resultado esperado
P1	Fechar a escalada do onboarding	Bloquear roleId/channelIds cross-guild, revalidar na conclusão e filtrar role.guildId no cálculo de permissões.
P2	Centralizar validação de referências tenant-scoped	Criar helpers assert*InGuild e aplicar em Space, badge, security policy, dynamic voice e novos módulos.
P3	Proteger ENDED com lease server-side	Não confiar em playbackOwnerRef; validar o detentor do lease no endpoint de controle.
P4	Fail-closed para PostgreSQL/Redis	Exigir secrets no compose/preflight e impedir Redis sem autenticação em produção.
P5	Adicionar testes de autorização cross-tenant	Fixtures com dois guilds e IDs trocados para cada rota IDOR/associação; rodar no CI.
P6	Adicionar secret scanning de histórico no CI	Executar gitleaks/trufflehog equivalente no clone completo; o ZIP auditado não contém .git e o histórico não pôde ser verificado nesta execução.

Limitações e condições de cobertura

- Auditoria estática: não foram executados ataques contra um ambiente de produção nem fuzzing dinâmico.
- O pacote auditado não contém .git. O working tree foi escaneado por marcadores/segregos e variáveis VITE, porém o histórico de commits não pôde ser confirmado.
- O repositório público informado (GabrielBosco/ginga) não pôde ser carregado pelo acesso de rede disponível nesta execução; por isso nenhuma afirmação de “histórico limpo” é feita.
- Endpoints públicos de mídia com etag/accessKey foram tratados como recursos tokenizados, não como IDOR, porque o segredo/etag faz parte do controle de acesso e foi verificado em código.

7. ISSUES PARA O GITHUB

Blocos completos em Markdown, prontos para copiar e colar. Achados sistêmicos relacionados foram agrupados para evitar spam.

Issue F1

--- ISSUE F1 ---

[Segurança] Onboarding aceita cargo de outro servidor e o aplica como permissão local

Labels sugeridas: `security`, `alta`

Descrição

A criação de opções do onboarding aceita roleId e channelId arbitrários. Na conclusão do onboarding, o roleId é gravado em GuildMemberCustomRole com o guildId da vítima sem confirmar que o cargo pertence ao mesmo servidor. O cálculo de permissões confia nessa associação e agrega as permissões do cargo estrangeiro.

Por que é explorável

Explorável por um usuário com manageServer em um servidor A que conheça o ID de um cargo customizado de outro servidor B. Ao configurar uma opção do onboarding com esse ID e fazer um membro concluir o fluxo, as permissões do cargo de B passam a ser consideradas no servidor A. O atacante não precisa ser dono de B, apenas conhecer um roleId válido; a condição prática mais simples é participar de ambos os servidores.

Evidência

- `apps/api/src/routes/v090.ts:76` - Entrada aceita roleId/channelIds sem validar o guild de cada referência.
- `apps/api/src/routes/v090.ts:81` - Conclusão grava roleId arbitrário em guildMemberCustomRole usando guildId do fluxo.
- `apps/api/src/v090Storage.ts:27` - FK do onboarding aponta para role por ID e channel_ids é TEXT[], sem vínculo de tenant.
- `apps/api/prisma/schema.prisma:297` - GuildMemberCustomRole referencia Guild e GuildCustomRole de forma independente.
- `apps/api/src/permissions.ts:250` - Cálculo agrega role.permissions das associações sem conferir role.guildId.

****apps/api/src/routes/v090.ts:76-76****

```
``text
76 | v090Router.post("/onboarding/questions/:questionId/options", requireAuth, asyncHandler(async (req, res) => {
    await ensureV090Storage();
    const q = routeParam(req.params.questionId, "questionId");
    qr = (await prisma.$queryRawUnsafe`any[]>(`SELECT guild_id FROM "GingaOnboardingQuestion" WHERE id=$1`, q))[0];
    if(!qr) throw new HttpError(404, "Pergunta nao encontrada");
    await requireGuildCapability(req.auth!.sub, qr.guild_id, "manageServer");
    const d = z.object({
      label: z.string().min(1).max(80),
      description: z.string().max(160).default(""),
      emoji: z.string().max(16).default(""),
      roleId: z.string().nullable().default(null),
      channelIds: z.array(z.string()).default([]),
      position: z.number().int().default(0)
    }).parse(req.body);
    id = randomUUID();
    await prisma.$executeRawUnsafe(`INSERT INTO "GingaOnboardingOption" (id, question_id, label, description, emoji, role_id, channel_ids, position) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)`, id, q, d.label, d.description, d.emoji, d.roleId, d.channelIds, d.position);
    res.status(201).json({ option: { id, ...d } });
  }));
...`
```

****apps/api/src/routes/v090.ts:81-81****

```
``text
81 | v090Router.put("/guilds/:guildId/onboarding/me", requireAuth, asyncHandler(async (req, res) => {
    await ensureV090Storage();
    const g = routeParam(req.params.guildId, "guildId");
    answers = z.record(z.string(), z.array(z.string()));
    .parse(req.body?.answers ?? {});
    await requireGuildMember(req.auth!.sub, g);
    const qs = await onboarding(g, roles = new Set<string>(), channels = new Set<string>());
    for(const q of qs.filter(q => q.enabled)) {
      const ids = answers[q.id] ?? [];
      if(q.required && ids.length) throw new HttpError(400, `Resposta: ${q.title}`);
      for(const id of ids) {
        const o = q.options.find((x: any) => x.id === id);
        if(!o) throw new HttpError(400, "Opcao invalida");
        if(o.roleId) roles.add(o.roleId);
        for(const c of o.channelIds) channels.add(c);
      }
      await prisma.$transaction(async tx => {
        for(const roleId of roles) await tx.guildMemberCustomRole.upsert({
          where: { guildId_userId_roleId: { guildId: g, userId: req.auth!.sub, roleId } },
          update: {},
          create: { guildId: g, userId: req.auth!.sub, roleId }
        });
        for(const c of channels) await tx.$executeRawUnsafe(`INSERT INTO "GingaChannelPreference" (user_id, channel_id, favorite, hidden, updated_at) VALUES ($1, $2, TRUE, FALSE, NOW()) ON CONFLICT (user_id, channel_id) DO UPDATE SET favorite=TRUE, hidden=FALSE, updated_at=NOW()`, req.auth!.sub, c);
        await tx.$executeRawUnsafe(`INSERT INTO "GingaOnboardingProgress" (guild_id, user_id, answers, completed_at, updated_at) VALUES ($1, $2, $3::jsonb, NOW(), NOW()) ON CONFLICT (guild_id, user_id) DO UPDATE SET answers=$3::jsonb, completed_at=NOW(), updated_at=NOW()`, g, req.auth!.sub, JSON.stringify(answers));
      });
      res.json({ completed: true, answers, favoriteChannelIds: [...channels] });
    }
  }));
...`
```

****apps/api/src/v090Storage.ts:27-29****

```
``text
27 | `CREATE TABLE IF NOT EXISTS "GingaOnboardingOption" (id TEXT PRIMARY KEY, question_id TEXT NOT NULL REFERENCES "GingaOnboardingQuestion"(id) ON DELETE CASCADE, label VARCHAR(80) NOT NULL, description VARCHAR(160) NOT NULL DEFAULT '', emoji VARCHAR(16) NOT NULL DEFAULT '', role_id TEXT REFERENCES "GuildCustomRole"(id) ON DELETE SET NULL, channel_ids TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[], position INTEGER NOT NULL DEFAULT 0, created_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP),
28 | `CREATE TABLE IF NOT EXISTS "GingaOnboardingProgress" (guild_id TEXT NOT NULL REFERENCES "Guild"(id) ON D`
```



```

    ELETE CASCADE,user_id TEXT NOT NULL REFERENCES "User"(id) ON DELETE CASCADE,answers JSONB NOT NULL DEFAULT '
    {}':::jsonb,completed_at TIMESTAMP(3),updated_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,PRIMARY KEY(
    guild_id,user_id))`,
29 | `CREATE TABLE IF NOT EXISTS "GingaOnboardingRoleGrant" (guild_id TEXT NOT NULL REFERENCES "Guild"(id) ON
    DELETE CASCADE,user_id TEXT NOT NULL REFERENCES "User"(id) ON DELETE CASCADE,role_id TEXT NOT NULL REFERENCE
    S "GuildCustomRole"(id) ON DELETE CASCADE,granted_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,PRIMARY
    KEY(guild_id,user_id,role_id))`,
...

```

Impacto

Escalada de privilégio entre tenants/servidores e quebra do isolamento de autorização.

Sugestão de correção

Validar roleId e cada channelId contra o guildId da pergunta antes de persistir; revalidar na aplicação do onboa-
rding; em effectiveGuildPermissionsForUser, carregar somente cargos cujo role.guildId seja igual ao guildId
da associação; adicionar invariante/constraint de aplicação para impedir associações cruzadas.

Critérios de aceite

- [] POST de opção rejeita roleId de outro guild com 400/404.
 - [] POST de opção rejeita channelIds de outro guild.
 - [] Conclusão do onboarding revalida os IDs persistidos antes de conceder cargos.
 - [] effectiveGuildPermissionsForUser ignora/rejeita qualquer associação cujo role.guildId divergir.
 - [] Teste automatizado cross-tenant prova que um cargo estrangeiro não concede capacidade no guild atual.
- FIM ISSUE F1 ---

Issue F2

--- ISSUE F2 ---

[Segurança] Atribuição de badge permite escrever em badge de outro servidor

Labels sugeridas: `security`, `média`

Descrição

A rota de atribuição exige manageRoles no guildId do path, mas insere badgeId e userId diretamente. Ela não conf-
irma que a badge pertence ao guild do path nem que o usuário alvo é membro dele.

Por que é explorável

Um moderador com manageRoles em A e conhecimento do ID de uma badge de B pode chamar a rota de A com badgeId de
B e associá-la a qualquer User existente. A remoção da mesma associação contém a checagem que está ausente n-
o PUT, confirmando a inconsistência.

Evidência

- `apps/api/src/routes/v090.ts:96` - PUT insere IDs sem vínculo; DELETE logo abaixo valida badge.id + guild_id.
- `apps/api/src/v090Storage.ts:34` - Assignment referencia badge_id/user_id, sem guild_id próprio ou constraint
composta.

apps/api/src/routes/v090.ts:96-98

```

`text
96 | v090Router.put("/guilds/:guildId/badges/:badgeId/members/:userId", requireAuth, asyncHandler(async (req, res)
    =>{const g=routeParam(req.params.guildId,"guildId");await requireGuildCapability(req.auth!.sub,g,"manageRole
    s");await prisma.$executeRawUnsafe(`INSERT INTO "GingaGuildBadgeAssignment"(badge_id,user_id,assigned_by) VA
    LUES($1,$2,$3) ON CONFLICT DO NOTHING`,routeParam(req.params.badgeId,"badgeId"),routeParam(req.params.userId
    ,"userId"),req.auth!.sub);res.status(204).end();}));
97 | v090Router.delete("/guilds/:guildId/badges/:badgeId", requireAuth, asyncHandler(async (req, res)=>{await ensu
    reV090Storage();const g=routeParam(req.params.guildId,"guildId"),b=routeParam(req.params.badgeId,"badgeId");
    await requireGuildCapability(req.auth!.sub,g,"manageRoles");const found=(await prisma.$queryRawUnsafe<any[]>
    (`SELECT id FROM "GingaGuildBadge" WHERE id=$1 AND guild_id=$2`,b,g))[0];if(!found)throw new HttpError(404,"
    Badge nao encontrada");await prisma.$executeRawUnsafe(`DELETE FROM "GingaGuildBadge" WHERE id=$1 AND guild_i
    d=$2`,b,g);emit(req,g);res.status(204).end();}));
98 | v090Router.delete("/guilds/:guildId/badges/:badgeId/members/:userId", requireAuth, asyncHandler(async (req, r
    es)=>{await ensureV090Storage();const g=routeParam(req.params.guildId,"guildId"),b=routeParam(req.params.bad
    geId,"badgeId"),u=routeParam(req.params.userId,"userId");await requireGuildCapability(req.auth!.sub,g,"manag
    eRoles");const found=(await prisma.$queryRawUnsafe<any[]>(`SELECT id FROM "GingaGuildBadge" WHERE id=$1 AND
    guild_id=$2`,b,g))[0];if(!found)throw new HttpError(404,"Badge nao encontrada");await prisma.$executeRawUnsa
    fe(`DELETE FROM "GingaGuildBadgeAssignment" WHERE badge_id=$1 AND user_id=$2`,b,u);emit(req,g);res.status(20
    4).end();}));
...

```

apps/api/src/v090Storage.ts:34-35

```

`text
34 | `CREATE TABLE IF NOT EXISTS "GingaGuildBadge" (id TEXT PRIMARY KEY,guild_id TEXT NOT NULL REFERENCES "Gui
    ld"(id) ON DELETE CASCADE,name VARCHAR(32) NOT NULL,icon VARCHAR(16) NOT NULL DEFAULT '',color VARCHAR(7) NO
    T NULL DEFAULT '#7c3cff',description VARCHAR(120) NOT NULL DEFAULT '',created_at TIMESTAMP(3) NOT NULL DEFAU

```



```

    LT CURRENT_TIMESTAMP)` ,
35 | `CREATE TABLE IF NOT EXISTS "GingaGuildBadgeAssignment" (badge_id TEXT NOT NULL REFERENCES "GingaGuildBad
    ge"(id) ON DELETE CASCADE,user_id TEXT NOT NULL REFERENCES "User"(id) ON DELETE CASCADE,assigned_by TEXT REF
    ERENCES "User"(id) ON DELETE SET NULL,assigned_at TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,PRIMARY KE
    Y(badge_id,user_id))`
...

```

Impacto

Alteração não autorizada de dados pertencentes a outro tenant; integridade de badges e perfis afetada.

Sugestão de correção

Buscar badge por {id: badgeId, guildId} antes do INSERT, exigir requireGuildMember(userId, guildId) para o alvo e manter a operação limitada à badge validada. Adicionar teste BOLA com IDs de guilds distintos.

Critérios de aceite

- [] PUT retorna 404/403 para badge de outro guild.
 - [] PUT rejeita userId que não seja membro do guild alvo.
 - [] Atribuição válida no mesmo guild continua funcionando.
 - [] Teste automatizado cobre badgeId estrangeiro e userId não membro.
- FIM ISSUE F2 ---

Issue F3

--- ISSUE F3 ---

[Segurança] Referências auxiliares v0.9 aceitam categorias/canais de outro tenant

****Labels sugeridas:**** `security`, `média`

Descrição

Várias estruturas auxiliares validam a permissão no recurso pai, porém não validam o tenant dos IDs referenciado s: conteúdo de Space, modLogChannelId e categoryId de template de voz dinâmica. As tabelas armazenam FKs sim ples por ID e não possuem vínculo composto com guild_id.

Por que é explorável

Exige capacidade legítima de administração no guild A e conhecimento de IDs de guild B. Em Space, IDs estrangeir os podem ser anexados a A. Em voz dinâmica, uma categoria de B pode ser salva no template e utilizada na cri ação de um Channel com guildId=A/categoryId=B; como canais sincronizados herdam permissões da category relac ionada, regras estruturais da categoria estrangeira podem influenciar o canal de A.

Evidência

- `apps/api/src/routes/v090.ts:54` - Space content insere categoryIds/channelIds após validar só o guild do Spac e.
- `apps/api/src/v090Storage.ts:18` - Join tables possuem FKs simples e não codificam guild_id.
- `apps/api/src/routes/v090.ts:86` - security-policy persiste modLogChannelId sem conferir o guild do canal.
- `apps/api/src/v090Storage.ts:30` - Policy/template referenciam Channel/Category apenas por ID.
- `apps/api/src/routes/v090.ts:90` - Template aceita categoryId e create usa esse valor para criar canal no guil d do path.
- `apps/api/prisma/schema.prisma:359` - Channel permite guildId e categoryId como FKs independentes.
- `apps/api/src/permissions.ts:420` - Canal sincronizado usa permissões/custom overrides/user overrides da categ ory relacionada.

****apps/api/src/routes/v090.ts:54-54****

```text

```

54 | v090Router.put("/spaces/:spaceId/content", requireAuth, asyncHandler(async (req, res) => {
 await ensureV090Stora
 ge(); const id = routeParam(req.params.spaceId, "spaceId"); rows = await prisma.$queryRawUnsafe<any[]>(`SELECT guil
 d_id FROM "GingaGuildSpace" WHERE id=$1`, id); if(!rows[0]) throw new HttpError(404, "Area nao encontrada"); cons
 t g = rows[0].guild_id; await requireGuildCapability(req.auth!.sub, g, "manageChannels"); const d = z.object({catego
 ryIds: z.array(z.string()).max(100), channelIds: z.array(z.string()).max(300)}).parse(req.body); await prisma.$t
 ransaction(async tx => { await tx.$executeRawUnsafe(`DELETE FROM "GingaGuildSpaceCategory" WHERE space_id=$1`, i
 d); await tx.$executeRawUnsafe(`DELETE FROM "GingaGuildSpaceChannel" WHERE space_id=$1`, id); for(const c of [
 ..new Set(d.categoryIds)]) await tx.$executeRawUnsafe(`INSERT INTO "GingaGuildSpaceCategory"(space_id, categor
 y_id) VALUES($1,$2)`, id, c); for(const c of [...new Set(d.channelIds)]) await tx.$executeRawUnsafe(`INSERT INTO
 "GingaGuildSpaceChannel"(space_id, channel_id) VALUES($1,$2)`, id, c); }); emit(req, g); res.json(d); });
...

```

**\*\*apps/api/src/v090Storage.ts:18-21\*\***

```text

```

18 | `CREATE TABLE IF NOT EXISTS "GingaGuildSpaceCategory" (space_id TEXT NOT NULL REFERENCES "GingaGuildSpace
    "(id) ON DELETE CASCADE, category_id TEXT NOT NULL REFERENCES "ChannelCategory"(id) ON DELETE CASCADE, PRIMARY
    KEY(space_id, category_id))`,
19 | `CREATE UNIQUE INDEX IF NOT EXISTS "GingaGuildSpaceCategory_category_key" ON "GingaGuildSpaceCategory"(ca
    tegory_id)`,
20 | `CREATE TABLE IF NOT EXISTS "GingaGuildSpaceChannel" (space_id TEXT NOT NULL REFERENCES "GingaGuildSpace"

```

```

    (id) ON DELETE CASCADE,channel_id TEXT NOT NULL REFERENCES "Channel"(id) ON DELETE CASCADE,PRIMARY KEY(space
    _id,channel_id))`,
21 | `CREATE UNIQUE INDEX IF NOT EXISTS "GingaGuildSpaceChannel_channel_key" ON "GingaGuildSpaceChannel"(chann
    el_id)`,
...
**apps/api/src/routes/v090.ts:86-86**
```text
86 | v090Router.put("/guilds/:guildId/security-policy",requireAuth,asyncHandler(async(req,res)=>{await ensureV
 090Storage();const g=routeParam(req.params.guildId,"guildId"),d=secSchema.parse(req.body);await requireGuild
 Capability(req.auth!.sub,g,"manageAutoMod");await prisma.$executeRawUnsafe(`INSERT INTO "GingaGuildSecurityP
 olicy"(guild_id,anti_raid_enabled,join_window_seconds,join_limit,quarantine_enabled,quarantine_minutes,new_a
 ccount_hours,block_external_links,block_invites,max_mentions,duplicate_limit,require_moderation_reason,auto_
 timeout_minutes,mod_log_channel_id,updated_at) VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,NOW())
 ON CONFLICT(guild_id) DO UPDATE SET anti_raid_enabled=$2,join_window_seconds=$3,join_limit=$4,quarantine_ena
 bled=$5,quarantine_minutes=$6,new_account_hours=$7,block_external_links=$8,block_invites=$9,max_mentions=$10
 ,duplicate_limit=$11,require_moderation_reason=$12,auto_timeout_minutes=$13,mod_log_channel_id=$14,updated_a
 t=NOW()`),g,d.antiRaidEnabled,d.joinWindowSeconds,d.joinLimit,d.quarantineEnabled,d.quarantineMinutes,d.newAc
 countHours,d.blockExternalLinks,d.blockInvites,d.maxMentions,d.duplicateLimit,d.requireModerationReason,d.au
 toTimeoutMinutes,d.modLogChannelId);res.json({policy:d});}));
...

Impacto
Quebra de invariantes de tenant, corrupção de relações e possibilidade de aplicar regras de categoria de outro s
ervidor a um canal recém-criado. O modLogChannelId atualmente é sobretudo um problema de integridade de conf
iguração, pois o uso de envio de log não foi encontrado no código auditado.

Sugestão de correção
Criar helpers de validação tenant-scoped (assertCategoryInGuild/assertChannelInGuild) e usá-los em todos os IDs
auxiliares antes de persistir. Para estruturas novas, preferir guardar guild_id e usar constraints/índices c
ompostos quando o modelo permitir. Na criação dinâmica, revalidar category_id antes do prisma.channel.create
.

Critérios de aceite
- [] Space rejeita categoria/canal de outro guild.
- [] security-policy rejeita modLogChannelId de outro guild.
- [] dynamic-voice template rejeita categoryId de outro guild.
- [] Criação dinâmica revalida a categoria persistida.
- [] Testes cross-tenant cobrem os três fluxos.
--- FIM ISSUE F3 ---

```

## Issue F4

```

--- ISSUE F4 ---
[Segurança] Ação ENDED do Ginga Music confia no playbackOwner do frontend

Labels sugeridas: `security`, `mídia`

Descrição
O frontend só envia ENDED quando playbackOwnerRef.current é verdadeiro. No backend, porém, a ação ENDED ignora m
usicAccess e exige apenas que o chamador seja membro do servidor. Um cliente modificado pode chamar o endpoi
nt diretamente e avançar a fila.

Por que é explorável
Qualquer membro autenticado do guild consegue consultar o estado/track atual e enviar POST /music/control com ac
tion=ENDED e expectedTrackId correto. Isso funciona mesmo quando musicAllowMembers=false e o cliente não pos
sui o lease oficial de playback.

Evidência
- `apps/web/src/components/GingaMusicPlayer.tsx:306` - Gate de playbackOwner existe somente no cliente antes do
 POST.
- `apps/api/src/routes/music.ts:178` - musicAccess é o helper server-side que restringe controle quando membros
 não podem controlar.
- `apps/api/src/routes/music.ts:444` - Qualquer membro pode ler o estado e obter o track atual.
- `apps/api/src/routes/music.ts:596` - ENDED usa apenas requireGuildMember e avança a fila quando expectedTrackI
 d confere.

apps/web/src/components/GingaMusicPlayer.tsx:306-317
```text
306 |   const reportEnded = useCallback((trackId: string) => {
307 |     if (!guildId || !trackId || endedRequestRef.current === trackId || !playbackOwnerRef.current) return;
308 |     endedRequestRef.current = trackId;
309 |     void api<{ state: MusicState }>(`/api/guilds/${guildId}/music/control`, {

```

```

310 |         method: "POST",
311 |         body: JSON.stringify({ action: "ENDED", expectedTrackId: trackId })
312 |     }).then((result) => acceptState(result.state)).catch(() => undefined).finally(() => {
313 |         window.setTimeout(() => {
314 |             if (endedRequestRef.current === trackId) endedRequestRef.current = "";
315 |         }, 700);
316 |     });
317 | }, [acceptState, guildId]);
...
**apps/api/src/routes/music.ts:178-184**
```text
178 | async function musicAccess(userId: string, guildId: string, requireManager = false) {
179 | const { membership, permissions } = await effectiveGuildPermissionsForUser(userId, guildId);
180 | const settings = await guildMusicSettings(guildId);
181 | const manager = membership.role === "OWNER" || membership.role === "ADMIN" || permissions.canManageServ
182 | er || permissions.canManageBots;
183 | if (requireManager && !manager) throw new HttpError(403, "Voce nao pode gerenciar o Ginga Music neste s
184 | eridor");
185 | if (!requireManager && !settings.musicAllowMembers && !manager) throw new HttpError(403, "Somente a mod
186 | eracao pode controlar a musica neste servidor");
187 | return { membership, permissions, settings, manager };
...
apps/api/src/routes/music.ts:444-453
```text
444 | musicRouter.get("/guilds/:guildId/music", requireAuth, asyncHandler(async (req, res) => {
445 |     const guildId = routeParam(req.params.guildId, "guildId");
446 |     const userId = req.auth!.sub;
447 |     await requireGuildMember(userId, guildId);
448 |     const settings = await guildMusicSettings(guildId);
449 |     const state = await runtimeFor(guildId);
450 |     res.json({
451 |         settings: publicSettings(settings),
452 |         state: publicRuntime(state)
453 |     });
...

```

Impacto

Manipulação não autorizada da fila e negação de serviço da reprodução compartilhada.

Sugestão de correção

Vincular ENDED ao lease de playback validado no servidor. Exigir clientId + lease/token não forjável e confirmar que o chamador possui o lease ativo para o guild/canal; alternativamente, mover avanço automático para um coordenador server-side. Não usar um booleano do frontend como fronteira de autorização.

Critérios de aceite

- [] ENDED sem lease ativo retorna 403/409.
 - [] ENDED de outro clientId não avança a fila.
 - [] Cliente detentor do lease continua conseguindo reportar fim da faixa.
 - [] musicAllowMembers=false não é burlado por chamada HTTP manual.
 - [] Teste de integração cobre chamada direta ao endpoint sem UI.
- FIM ISSUE F4 ---

Issue F5

--- ISSUE F5 ---

[Segurança] Defaults públicos de PostgreSQL/Redis podem virar credenciais reais se init.sh for ignorado

****Labels sugeridas:**** `security`, `baixa`

Descrição

.env.example contém senhas públicas CHANGE_ME para PostgreSQL e Redis. O compose aceita esses valores e permite Redis sem autenticação quando REDIS_PASSWORD fica vazio. A validação de startup rejeita CHANGE_ME para JWT/LiveKit, mas não falha para credencial de banco/Redis.

Por que é explorável

Condição necessária: operador copiar .env.example ou limpar REDIS_PASSWORD sem executar scripts/init.sh, ou expor essas redes/portas por configuração adicional. No compose padrão PostgreSQL/Redis não têm portas publicadas no host, reduzindo a exposição para rede interna/lateral. scripts/init.sh mitiga o cenário normal ao gerar segredos aleatórios, mas é uma convenção e não um controle fail-closed.

Evidência

- `.env.example:7` - Placeholders públicos podem ser copiados como credenciais de runtime.

```

- `docker-compose.yml:10` - PostgreSQL recebe POSTGRES_PASSWORD diretamente do ambiente.
- `docker-compose.yml:22` - Redis aceita REDIS_PASSWORD vazio e inicia sem --requirepass nesse caso.
- `docker-compose.yml:89` - API constrói DATABASE_URL com POSTGRES_PASSWORD fornecido.
- `apps/api/src/config.ts:40` - Fail-closed cobre JWT/LiveKit, mas não PostgreSQL/Redis.
- `scripts/init.sh:34` - Mitigação existente: bootstrap gera senhas/secrets aleatórios.

**env.example:7-8**
```text
7 | POSTGRES_PASSWORD=CHANGE_ME_DATABASE_PASSWORD
8 | REDIS_PASSWORD=CHANGE_ME_REDIS_PASSWORD
```

**docker-compose.yml:10-12**
```text
10 | POSTGRES_DB: ${POSTGRES_DB}
11 | POSTGRES_USER: ${POSTGRES_USER}
12 | POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

**docker-compose.yml:22-33**
```text
22 | redis:
23 | image: redis:7-alpine
24 | restart: unless-stopped
25 | security_opt:
26 | - no-new-privileges:true
27 | environment:
28 | REDIS_PASSWORD: ${REDIS_PASSWORD:-}
29 | command: ["/bin/sh", "-ec", 'if [-n "$$REDIS_PASSWORD"]; then exec redis-server --appendonly yes --requirepass "$$REDIS_PASSWORD"; else exec redis-server --appendonly yes; fi']
30 | volumes:
31 | - redis_data:/data
32 | healthcheck:
33 | test: ["CMD-SHELL", 'if [-n "$$REDIS_PASSWORD"]; then redis-cli --no-auth-warning -a "$$REDIS_PASSWORD" ping; else redis-cli ping; fi']
```

## Impacto
Credenciais previsíveis/ausência de senha em serviços internos se o bootstrap seguro for contornado.

## Sugestão de correção
Tornar POSTGRES_PASSWORD e REDIS_PASSWORD obrigatórios no compose com ${VAR:?required}; iniciar Redis sempre com --requirepass; adicionar preflight/startup que rejeite vazio/CHANGE_ME para credenciais de infraestrutura.
Manter init.sh como conveniência, não como única barreira.

## Critérios de aceite
- [ ] docker compose falha antes de subir se POSTGRES_PASSWORD estiver vazio/placeholder.
- [ ] docker compose falha se REDIS_PASSWORD estiver vazio/placeholder.
- [ ] Redis sempre inicia com autenticação no perfil de produção.
- [ ] Preflight documenta e testa placeholders proibidos.
- [ ] Instalação via init.sh continua gerando valores aleatórios.
--- FIM ISSUE F5 ---

```